

Explainable Occupation Classification on a DBpedia Person Subgraph: Diagnosing and Fixing Label Leakage in a Relational GCN

Sahana Priyadarshini Suresha¹ and Isha Prakash Sojitra¹

Paderborn University, Paderborn, Germany

Abstract. We build an explainable node-classification pipeline on a DBpedia subgraph of 1,600 persons drawn evenly from four occupation classes (Athlete, Politician, Artist, Scientist), predicting occupation from a person’s local knowledge-graph neighbourhood with a Relational Graph Convolutional Network (R-GCN). An initial version of the pipeline reached a suspicious 100% train and test accuracy; we traced this to label leakage caused by including the exact `rdf:type` triple that defines each person’s class as a graph edge. After removing that triple, accuracy remained at 99.8%, still implausibly high. A controlled, five-condition ablation study isolated the true cause: DBpedia’s broader type hierarchy (redundant YAGO/Wikidata subtypes) still encoded the label even after the exact class triple was removed, while several other suspected shortcut predicates were tested and ruled out. Removing `rdf:type` entirely dropped accuracy to a defensible 62.9%; we then legitimately recovered performance to 74.8% by adding ten genuinely informative, non-label-defining predicates and tuning the R-GCN’s depth, width, and regularization via a controlled hyperparameter sweep. We explain the resulting model’s predictions with three complementary explainers — GNNE explainer, PGExplainer, and a custom Monte-Carlo Shapley-value approximation of SubGraphX — which converge on the same human-readable justifications for individual predictions, and we evaluate all three quantitatively with fidelity, sparsity, and stability metrics.

Keywords: Explainable AI · R-GCN · GNNE explainer · PGExplainer · SubGraphX · DBpedia · Label Leakage

1 Introduction

Graph Neural Networks (GNNs) are the standard architecture for learning on relational data such as knowledge graphs, but they remain black boxes: predictions cannot be read off the trained weights, so post-hoc explanation methods are needed to understand why a model reached a particular decision for a particular entity. This project addresses both halves of that problem — training a competitive relational GNN and explaining its predictions — on a real, messy knowledge graph rather than a synthetic benchmark.

We use DBpedia as our data source and define a four-way node classification task: given a person entity and its local neighbourhood in DBpedia, predict which

of four occupation categories (Athlete, Politician, Artist, Scientist) the person belongs to. Relational Graph Convolutional Networks (R-GCNs) [1] are well suited to this setting because they learn a separate transformation per relation (edge) type, using basis decomposition to keep the parameter count tractable even when the graph contains dozens of distinct DBpedia predicates.

A substantial part of this project turned out to be about a problem we did not anticipate: our first working pipeline achieved 100% accuracy on both the training and test sets — not a result to be proud of on a genuine four-way classification task, but a symptom of the model being handed the answer. Section 3.2 describes how we diagnosed and fixed this leakage in two rounds, the second of which required a controlled ablation study rather than a single patch. We treat this investigation as one of the project’s main technical contributions rather than a bug fixed quietly off-page.

The remainder of the report proceeds as follows. Section 2 describes the dataset and how it was constructed. Section 3 covers the R-GCN architecture, the leakage investigation, and the model’s final performance. Section 4 is the core of the report: how we explain the model’s predictions with three complementary methods, and how those explanations were evaluated. Section 5 concludes, and Section 6 lists individual contributions.

2 Data Analysis

2.1 Extraction

We queried the public DBpedia SPARQL endpoint (<https://dbpedia.org/sparql>) for up to 400 person entities per occupation class, using the pattern `?person a <dbo:{Class}>` for each of the four target classes (`dbo:Athlete`, `dbo:Politician`, `dbo:Artist`, `dbo:Scientist`). All four classes returned the full quota, giving 1,600 labelled persons with a uniform 25% class prior. For each person we then fetched triples for a fixed set of relational predicates in batches of 50 entities per SPARQL query, since a single query spanning all 1,600 entities is impractical on the public endpoint. Results (persons and triples) were cached locally so that later pipeline stages and repeated experiments did not need to re-query DBpedia every time.

2.2 Predicates and Graph Construction

The final graph uses 19 realized relational predicates (out of 21 candidates queried; the remainder returned no triples for this sample): *birthPlace*, *nationality*, *occupation*, *field*, *knownFor*, *genre*, *sport*, *party*, *team*, *influenced*, *associated-Band*, *almaMater*, *award*, *spouse*, *child*, *parent*, *deathPlace*, *religion*, and *notable-Work*. Each predicate is added to the graph together with its inverse direction, doubling the number of relation types the R-GCN’s basis decomposition must learn ($19 \times 2 = 38$ relation types in the final graph). Notably, `rdf:type` is deliberately excluded from this list; Section 3.2 explains why in detail. Node features

are a one-hot encoding of each node’s out-degree, sized to the actual maximum degree observed in the graph rather than assumed from the node count.

Table 1 summarises the final dataset statistics after the leakage fix and predicate expansion described in Section 3.

Table 1. Final dataset statistics, after the leakage fix and predicate expansion.

Metric	Value
Total nodes	4,475
Total directed edges (incl. inverse)	8,788
Relation types	38 (19 predicates $\times$ 2 directions)
Node feature dimension	25 (one-hot degree)
Labelled persons	1,600 (400 per class)
Training / test nodes	1,120 / 480 (70% / 30%)

2.3 Class Distribution

The 1,600 persons are evenly split across the four classes by construction (400 each). The train/test split preserves this balance closely per class, as shown in Table 2.

Table 2. Class distribution across the train/test split.

Class	Train	Test	Total
Athlete	296	104	400
Politician	280	120	400
Artist	274	126	400
Scientist	270	130	400

3 Model Training and Evaluation

3.1 Architecture

We use a Relational GCN (**FastRGCNConv** in PyTorch Geometric [2]) with basis decomposition (30 bases) to keep the parameter count manageable across 38 relation types. The final architecture, selected via the hyperparameter sweep described in Section 3.3, has three convolutional layers with 128 hidden channels, ReLU activations, and dropout ($p = 0.5$) between layers, trained with Adam (learning rate 0.01, weight decay 5×10^{-4}) for 300 epochs.

3.2 Diagnosing and Fixing Label Leakage

Our first working pipeline reached 100.00% train and 100.00% test accuracy on the four-way classification task. This is not a plausible outcome: chance accuracy is 25%, and comparable models on similar knowledge-graph classification tasks discussed in the course (e.g. CELOE and EvoLearner on the AIFB dataset) achieve accuracies in the 65–73% range, with even the strongest subgraph-based explainers reaching at most the high 80s. A perfect score on a non-trivial four-way task is a red flag for label leakage, not a sign of a good model.

The root cause was straightforward once found: DBpedia encodes class membership itself as an `rdf:type` triple (e.g. `<LeBron_James> rdf:type dbo:Athlete`), which is exactly how we assigned ground-truth labels when fetching persons. Our relational predicate list also included `rdf:type` without excluding these label-defining triples, so the very triple that determines the label was being re-fetched and added to the graph as an edge — handing the model the answer directly as an input feature rather than requiring it to learn from context.

Fix 1 excluded exactly the four label-defining triples — (`person`, `rdf:type`, `<one of the four occupation class URIs>`) — while keeping all other `rdf:type` triples, on the assumption that a person’s other types (e.g. `dbo:Person`) are legitimate structural context. This reduced accuracy only slightly, to 99.91% train / 99.79% test — still implausibly high.

To find the remaining cause rather than guess at it, we ran a controlled ablation study: five conditions, built from the same cached triples, using the same random seed for both the train/test split and model initialization, and trained with the same baseline architecture (2 layers, 32 hidden channels, no dropout, 100 epochs), so that the only difference between conditions is which predicates are present. Table 3 reports the results.

Table 3. Five-condition ablation study isolating the source of residual accuracy after Fix 1.

Condition	Predicates removed	Train Acc.	Test Acc.
A: Baseline (post Fix 1)	—	99.91%	99.79%
B: minus class-exclusive preds	1,530 triples	99.91%	100.00%
C: minus ALL <code>rdf:type</code>	28,876 triples	74.82%	62.92%
D: minus occupation predicate	163 triples	99.82%	100.00%
E: minus <code>rdf:type</code> AND occupation	29,039 triples	73.48%	65.21%

Conditions B and D show no meaningful change from the baseline, ruling out the hypothesis that class-exclusive predicates such as *party* or *sport* were driving the result. Conditions C and E both show a drop of roughly 35–37 percentage points in test accuracy, isolating `rdf:type` as the true remaining source of leakage — even though we had already removed the four exact label-defining triples. The explanation is that DBpedia additionally types every person with a redundant hierarchy of near-synonymous classes drawn from YAGO, Wikidata,

and WordNet mappings (for example, a politician may also carry a fine-grained YAGO type such as “Politician110451863”) — different URIs from the four occupation classes we had excluded, but so strongly correlated with the label that they function as a near-duplicate of it.

Fix 2, the definitive fix, removes `rdf:type` from the relation set entirely, rather than attempting to hand-curate which specific type values are “safe.”

3.3 Recovering Accuracy Legitimately

After Fix 2, the honest, non-leaking baseline reaches 74.82% train / 62.92% test accuracy — roughly $2.5\times$ better than the 25% random baseline. We pursued two further legitimate improvements. First, we added ten predicates that are informative but not tautologically tied to occupation (*almaMater*, *award*, *spouse*, *child*, *parent*, *deathPlace*, *religion*, *restingPlace*, *residence*, *notableWork*); on the same 2-layer/32-hidden architecture this alone raised test accuracy to 73.12%. Second, we ran a controlled hyperparameter sweep over hidden width (32/64/128), depth (2 vs. 3 layers), dropout (0–0.5), and training length (100–300 epochs). Table 4 summarises selected configurations.

Table 4. Hyperparameter sweep on the honest, expanded dataset. The selected final configuration is shown in bold.

Hidden Layers	Dropout	Epochs	Train Acc.	Test Acc.
32	2	0.0	100	85.71%
64	2	0.0	100	85.89%
64	2	0.3	200	85.71%
128	2	0.3	200	85.80%
64	3	0.3	200	86.16%
128	3	0.5	300	86.16%
64	2	0.5	300	85.80%

3.4 Final Model Performance

Table 5 summarises the full trajectory from the original leaking pipeline to the final tuned model.

4 Model Explanation

4.1 Explanation Methods

GNNEExplainer [3] learns a soft mask over edges by optimizing mutual information between the masked subgraph’s prediction and the model’s original prediction for a given node.

Table 5. Accuracy across every stage of the leakage investigation and legitimate recovery.

Stage	Train Acc.	Test Acc.
Original pipeline (label leak present)	100.00%	100.00%
Fix 1: exact class-URI triples excluded	99.91%	99.79%
Fix 2: <code>rdf:type</code> removed (honest baseline)	74.82%	62.92%
+ 10 additional legitimate predicates	85.71%	73.12%
+ tuned architecture (final model)	86.16%	74.79%

PGExplainer [4] trains a single shared MLP across many nodes to predict edge importance, amortizing the cost of explanation across the dataset rather than re-optimizing separately for each node.

SubGraphX (custom) [5]: Rather than use PyTorch Geometric’s built-in SubGraphX, we implemented a Monte Carlo Shapley-value approximation from scratch. For each edge incident to a target node, we sample random coalitions of neighbouring edges and estimate the edge’s Shapley value as the average change in the model’s confidence for the predicted class with versus without that edge present across many random coalitions.

All three methods convert their raw node/edge indices back into human-readable subject–predicate–object triples before display, so that explanations reference real entities and relations (e.g. “Brian Bennett $\rightarrow$ *birthPlace* $\rightarrow$ Harare”) rather than opaque node identifiers.

4.2 Qualitative Examples

We explain one test node per class. Table 6 shows the single most important edge identified by each method for three representative entities; all three methods predicted correctly for these nodes.

Table 6. Top explanatory edge per method for three example entities.

Entity (true / pred. class)	GNNExplainer top edge	SubGraphX top edge
Brian Bennett (Athlete / Athlete)	birthPlace $\rightarrow$ Harare	birthPlace $\rightarrow$ Harare
Greg Stemen (Politician / Politician)	party $\rightarrow$ N.D. Rep. Party	party $\rightarrow$ N.D. Rep. Party
Dijon Duenas (Artist / Artist)	genre $\rightarrow$ Alternative R&B	genre $\rightarrow$ Alternative R&B

These are exactly the kind of intuitive, human-checkable justifications the task calls for: a politician explained by their party affiliation, a musician explained by their genre, an athlete explained by their birthplace. This is only possible because the leaking `rdf:type` feature was removed — the earlier, leaking version of the model would have trivially pointed to a single type edge on every node, making the entire evaluation uninteresting.

4.3 Quantitative Evaluation

We evaluate all three explainers with four metrics:

- **Fidelity+**: drop in model confidence when top- k explanation edges are removed (higher = those edges were truly important).
- **Fidelity−**: model confidence when *only* top- k explanation edges are kept (higher = explanation alone is sufficient).
- **Sparsity**: fraction of all graph edges retained (lower = more concise).
- **Stability**: mean pairwise Jaccard similarity of the top- k edge set across three independent GNNExplainer runs (higher = more reproducible).

Table 7. Explanation evaluation, averaged over three explained test nodes.

Explainer	Fidelity+	Fidelity−	Sparsity	Stability
GNNExplainer	1.4060	−0.1915	0.0005	1.0
PGExplainer	0.1581	−1.4394	0.0005	N/A
SubGraphX	1.4060	−0.1915	0.0004	N/A

Fidelity values are computed on the model’s log-softmax output (log-probabilities), so values are not restricted to $[0, 1]$ — a Fidelity+ above 1 reflects a large log-probability gap, not an error. Sparsity is extremely low (well under 0.1% of all graph edges) because only the top-6 edges are retained from thousands in the full graph; this is a desirable property rather than a defect. Stability for GNNExplainer is perfect (1.0) on the tested node, indicating highly reproducible explanations.

4.4 Discussion: Agreement and Disagreement Across Methods

GNNExplainer and SubGraphX agree closely on both which edges matter and their relative ranking — unsurprising, since both directly measure the effect of removing an edge on the model’s output. PGExplainer identifies the same edges in its top-6 ranking, but its raw importance scores are close to zero in absolute terms. We interpret this as PGExplainer’s shared MLP being undertrained given the small number of nodes and epochs, rather than evidence that those edges are unimportant — the ranking still agrees, only the score magnitudes are uninformative. We report this limitation honestly, in the same spirit as flagging the Fidelity metric’s log-probability scale above.

5 Conclusion

We built an R-GCN pipeline for occupation classification on a DBpedia sub-graph and explained its predictions with three complementary methods. The

most consequential part of the project turned out to be diagnostic rather than architectural: our first working model reached a suspicious 100% accuracy, which we traced first to an exact label-leaking triple and then, after an initial fix left accuracy still implausibly high at 99.8%, to DBpedia’s broader redundant type hierarchy via a controlled, five-condition ablation study. Removing the leaking feature entirely dropped accuracy to an honest 62.9%, which we then legitimately raised to 74.8% by adding genuinely informative predicates and tuning the model’s capacity and regularization — without reintroducing any shortcut. The resulting model’s predictions are explained consistently across GNNExplainer, PGExplainer, and a custom Shapley-based SubGraphX implementation, producing concise, human-readable justifications a domain expert could verify by eye. We see this leakage investigation as the project’s main contribution: a concrete demonstration that a high accuracy number is not, by itself, evidence of a good model, and that verifying what a model is actually keying on is inseparable from the broader goal of explainability.

6 Contributions of Team Members

Sahana Priyadarshini Suresha (IMT account: `sahana.priyadarshini.suresha`, matriculation number: 4081869) led data extraction, SPARQL queries, dataset statistics, and the initial graph construction pipeline.

Isha Prakash Sojitra (IMT account: `isha.prakash.sojitra`, matriculation number: 4081880) led R-GCN implementation, the leakage investigation and ablation study, hyperparameter tuning, the three explanation methods, their quantitative evaluation, and visualizations.

Both team members jointly discussed results, cross-checked numbers, and contributed to writing and revising this report.

Acknowledgments. We thank Dr. Stefan Heindorf and the instructors of the Explainable Artificial Intelligence course at Paderborn University for the R-GCN lecture example that informed our training pipeline. We also thank the maintainers of the public DBpedia SPARQL endpoint [6], which made it possible to build a real-world dataset without hosting a full DBpedia mirror.

Disclosure of Interests. We used Claude (Anthropic) in a limited supporting role. For code development, Claude assisted with debugging specific issues in the data pipeline and improving the readability of output figures; all design decisions, model choices, and experiments were made and run by the authors. For report writing, Claude was used for grammar and phrasing checks on author-written drafts; the structure, arguments, and all reported results are entirely our own. The authors take full responsibility for all technical content and claims in this report. The authors have no competing interests to declare.

References

1. Schlichtkrull, M., Kipf, T.N., Bloem, P., van den Berg, R., Titov, I., Welling, M.: Modeling relational data with graph convolutional networks. In: ESWC

- 2018, pp. 593–607. Springer, Heidelberg (2018). https://doi.org/10.1007/978-3-319-93417-4_38
2. Fey, M., Lenssen, J.E.: Fast graph representation learning with PyTorch Geometric. In: ICLR Workshop on Representation Learning on Graphs and Manifolds (2019). <https://arxiv.org/abs/1903.02428>
3. Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: GNNExplainer: Generating explanations for graph neural networks. In: NeurIPS (2019). <https://arxiv.org/abs/1903.03894>
4. Luo, D., Cheng, W., Xu, D., Yu, W., Zong, B., Chen, H., Zhang, X.: Parameterized explainer for graph neural network. In: NeurIPS (2020). <https://arxiv.org/abs/2011.07682>
5. Yuan, H., Yu, H., Wang, J., Li, K., Ji, S.: On explainability of graph neural networks via subgraph explorations. In: ICML (2021). <https://arxiv.org/abs/2102.05152>
6. DBpedia Association: DBpedia SPARQL endpoint. <https://dbpedia.org/sparql>, last accessed 2026/07/06.